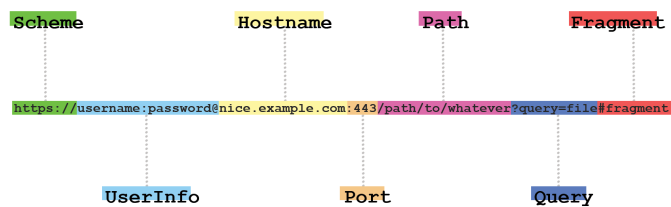


Protocole HTTP

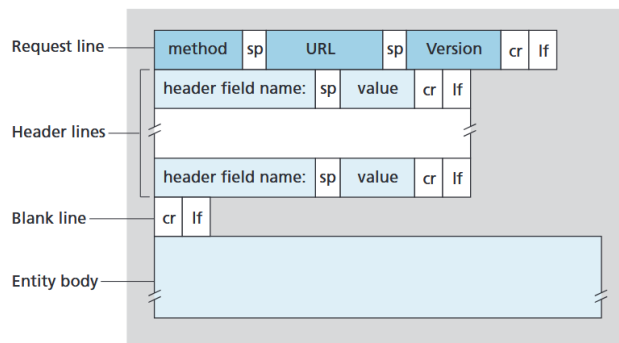
David Darras

Date de création : 31/12/2024
Dernière modification : 01/01/2025

- Une **page Web** est constituée d'objets (ou ressources Web) tels que des fichiers HTML, CSS, JS, JPEG, etc., adressables par une URL. L'URL `http://www.someSchool.edu/someDepartment/picture.gif` se compose du nom d'hôte du serveur qui héberge l'objet : `www.someSchool.edu`, et du chemin d'accès de l'objet `/someDepartment/picture.gif`. La demande d'une page Web se fait du côté client, via un navigateur tel que *Mozilla* ou *Chrome*, qui initie la connexion avec un serveur utilisant généralement *Apache* sur le port 80, avec une adresse IP fixe obtenue via le DNS.



- **HTTP** (*HyperText Transfer Protocol*) est un protocole de la couche application au cœur du Web, qui repose sur la couche de transport TCP, et est décrit dans les [RFC 1945], [RFC 7230] et [RFC 7540]. Il définit la manière dont les clients demandent une page Web et comment les serveurs répondent. C'est un protocole dit **sans état**, car le serveur ne sauvegarde pas en mémoire les fichiers déjà envoyés aux clients. Enfin, il existe plusieurs versions : HTTP/1.0, utilisé depuis les années 90, suivi par HTTP/1.1, puis HTTP/2. HTTP/1.0 utilisait des connexions non persistantes. Pour télécharger 100 objets, on devait ouvrir 100 connexions TCP (série ou parallèle). Par défaut, HTTP/1.1 utilise une connexion persistante. On gagne en temps, car il n'est plus nécessaire de refaire un *three-way handshake* pour chaque objet, et on a plus de mémoire libre, car un seul tampon TCP est alloué.
- **Requête HTTP** (envoyée par le client via un navigateur)



Remarques :

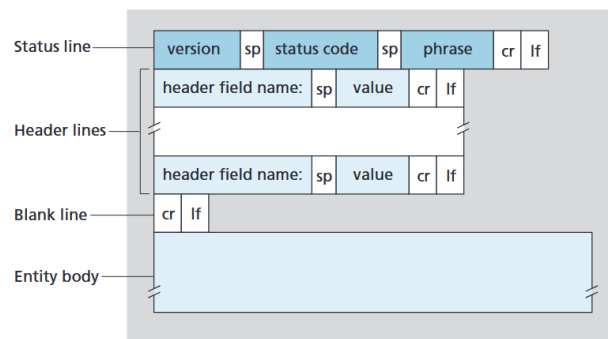
- On peut remplir un formulaire en utilisant GET et en mettant les informations fournies dans l'URL (ex : `www.somesite.com/animalsearch?monkeys&bananas`).
- Le corps de la requête est présent uniquement avec les méthodes POST, PUT, PATCH et DELETE.
- CR est un retour chariot '\r', LF un retour à la ligne '\n', et SP un espace ' '.

```
1 GET /somedir/page.html HTTP/1.1      # Ligne de requête : [Méthode] [URI] [Version HTTP]
2 Host: www.someschool.edu              # Lignes d'en-tête : L'hôte est utile pour les serveurs proxy
3 Connection: close                     # Pas de connexion persistante, à l'opposé de Keep-Alive
4 Accept-Language: fr                   # Préférer une version française si possible, sinon c'est la version par défaut qui est envoyée
5 User-Agent: Mozilla/5.0                # Type de navigateur
6                                     # On peut avoir différentes versions d'un même objet
```

Méthodes :

Méthode	Commentaire
GET	Le navigateur demande un objet au serveur identifié par un URI.
POST	Le navigateur envoie un formulaire au serveur.
HEAD	Agit comme GET, sauf que le serveur n'envoie pas d'objet (utile pour le débogage).
PUT	Téléverse un objet vers un répertoire spécifique du serveur Web.
DELETE	Supprime un objet spécifique sur un serveur.
TRACE	Permet de suivre un message à travers les serveurs proxy jusqu'au serveur final.
OPTIONS	Détermine quelles méthodes peuvent être utilisées sur un serveur.

• Réponse HTTP (envoyée par un serveur) :



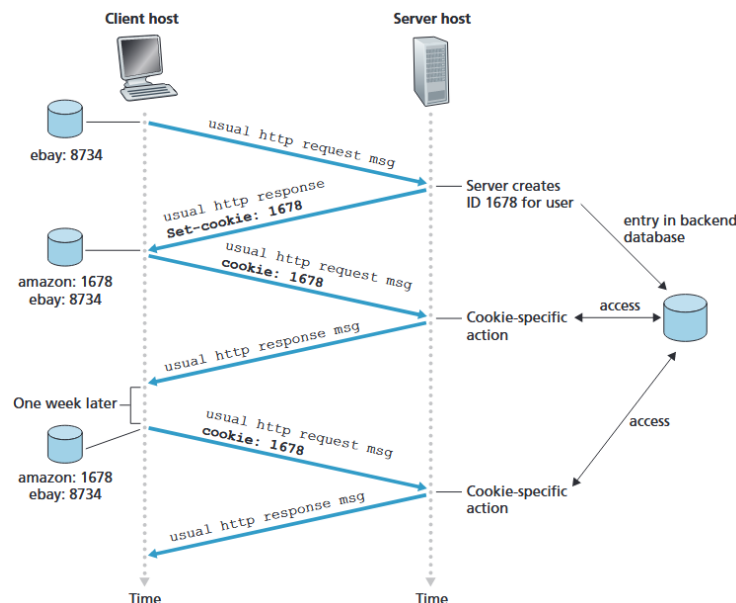
Les **codes d'état** les plus connus sont : 200 OK pour une requête réussie et lorsque les informations ont été envoyées, 301 Moved Permanently qui fournit la nouvelle URL via Location: pour un objet définitivement déplacé, 400 Bad Request si le serveur n'a pas compris la requête, 404 Not Found si l'objet n'existe pas sur le serveur, 505 HTTP Version Not Supported, etc.

```

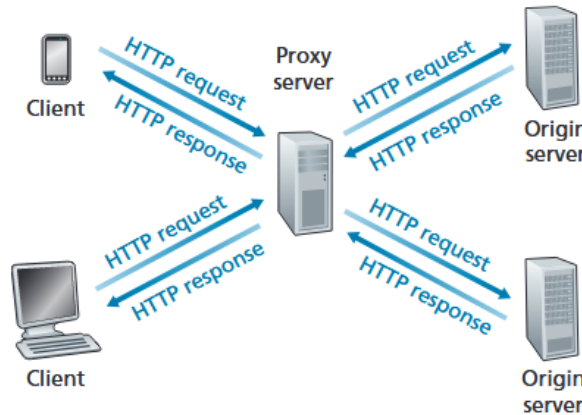
1 HTTP/1.1 200 OK # Ligne de statut initiale : [Version HTTP] [Code d'état] [Message d'état]
2 Connection: close # Lignes d'en-tête - ici le serveur ferme la connexion
3 Date: Tue, 18 Aug 2015 15:44:04 GMT # Date à laquelle la réponse (le paquet HTTP) a été envoyée depuis le serveur
4 Server: Apache/2.2.3 (CentOS) # Analogue à User-Agent, identifie le serveur
5 Content-Length: 6821 # Taille de l'objet en octets
6 Content-Type: text/html # Type d'objet
7 Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT # Date à laquelle l'objet en lui-même a été modifié (ou créé)
8 (data data data data data ...) # Utile pour la mise en cache côté client et serveur avec les proxys
9 # Corps de l'entité
10 # Contient l'objet demandé par GET, vide sinon

```

- Les serveurs sont sans état. Pour qu'un site puisse suivre un utilisateur, on utilise des **cookies** transmis via HTTP et une **base de données**, comme *MySQL*, côté serveur pour sauvegarder les données utilisateur. Par exemple, si je vais sur Amazon pour la première fois (anonymement, sans compte), je serai ajouté à la base de données avec un cookie qui sera envoyé à mon navigateur via HTTP, et ce dernier le stockera dans un fichier. À chaque nouvelle requête envoyée à Amazon, je transmettrai mon cookie via le champ homonyme pour qu'Amazon suive mon activité, mon panier, etc.



- Un **cache Web** (ou *caching proxy*) est une entité qui satisfait les requêtes HTTP au nom d'un serveur Web d'origine. Il conserve des copies des objets récemment demandés sur son propre support de stockage. Par exemple, si le client demande un objet et que celui-ci est dans le cache, il répond directement ; sinon, il le demande au serveur d'origine et le stocke selon la politique choisie. Il permet de réduire le temps de réponse à une requête client ainsi que le trafic sur Internet. Pour éviter une incohérence au niveau du cache, HTTP utilise le **GET conditionnel**, qui repose sur le champ **If-Modified-Since** : pour vérifier que l'objet dans le cache est le même que celui sur le serveur d'origine et qu'il n'a pas été modifié entre-temps. Si les dates correspondent, le serveur envoie **304 Not Modified** sans l'objet ; sinon, il renvoie l'objet que le cache mettra à jour.



- HTTP/2 réduit les latences en compressant les en-têtes. De plus, les objets sont envoyés de manière entrelacée pour réduire le délai perçu par l'utilisateur (on parle de **multiplexage**) et permet donc de résoudre le *problème de HOL*. Il permet également, pour les requêtes parallèles, d'affecter une priorité aux objets afin de permettre aux développeurs d'optimiser leur site. Enfin, le serveur peut analyser le HTML demandé et envoyer directement les objets attendus, comme des images, des scripts, etc., au lieu d'attendre un GET du client (qui sera forcément effectué pour le rendu de la page Web).

Exemple

```

1 C:\Users\David>telnet www.usa.gov 80
2 Trying 52.222.149.97...
3 Connected to www.usa.gov.
4 Escape character is '^]'.
5 GET /travel HTTP/1.1
6 Host: www.usa.gov
7
8 HTTP/1.1 301 Moved Permanently
9 Server: CloudFront
10 Date: Wed, 01 Jan 2025 10:31:30 GMT
11 Content-Type: text/html
12 Content-Length: 167
13 Connection: keep-alive
14 Location: https://www.usa.gov/travel
15 X-Cache: Redirect from cloudfront
16 Via: 1.1 7d935e83126b0b85ded112b940f9c85c.cloudfront.net ...
17 X-Amz-Cf-Pop: CDG52-P1
18 X-Amz-Cf-Id: _t0igCEf6XuWqhVjbvxjDTaZqdLQ6UoRCNQ1Utt4-_V...
19
20 <html>
21 <head><title>301 Moved Permanently</title></head>
22 <body>
23 <center><h1>301 Moved Permanently</h1></center>
24 <hr><center>CloudFront</center>
25 </body>
26 </html>

```

Références

